

Documentación Técnica - Validador de Monetización

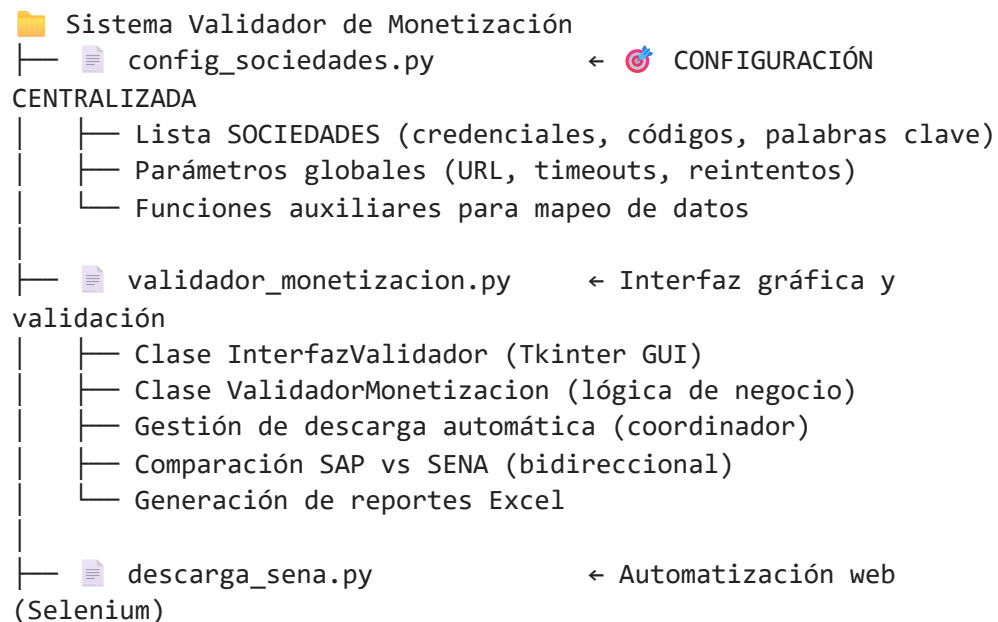
Contenido

1. Arquitectura del Sistema
 2. Configuración Centralizada
 3. Descarga Automática SENA
 4. Dependencias y Conexiones
 5. Clases Principales
 6. Estructura de Datos
 7. Funciones Core
 8. Interfaz Gráfica
 9. Flujo de Procesamiento
 10. Generación de Reportes
 11. Mantenimiento y Extensibilidad
 12. Troubleshooting
 13. Requisitos del Sistema
-

Arquitectura del Sistema

Componentes Principales

El sistema está compuesto por tres módulos principales:




```
{
    "nombre": "Bancolombia",
    "usuario": "890903938",
    "contrasena": "Preseleccion2023",
    "codigo_interno": "1000",
    "palabras_clave": ["bancolombia"]
}
```

Funciones Auxiliares

obtener_mapeo_codigos()

Propósito: Genera diccionario para convertir códigos internos a nombres

Retorna: {'1000': 'Bancolombia', '3000': 'Fiduciaria', ...}

Uso: Traducción de códigos en reportes y validación

obtener_sociedades_palabras_clave()

Propósito: Genera diccionario para detección automática de sociedades

Retorna: {'bancolombia': ('1000', 'Bancolombia'), ...}

Uso: Identificación de sociedad por nombre de archivo

obtener_opciones_dialogo()

Propósito: Genera lista para diálogos de selección

Retorna: [('Bancolombia', '1000'), ('Fiduciaria', '3000'), ...]

Uso: Generación dinámica de botones en interfaz

Parámetros Globales

```
URL_SENA =
"https://caprendizaje.sena.edu.co/sgva/SGVA_Disenio/pag/login.aspx"
MAX_REINTENTOS_DESCARGA = 3      # Reintentos por sociedad
TIMEOUT_DESCARGA = 60           # Segundos para completar descarga
```

Escalabilidad

Para agregar una nueva sociedad:

1. Editar `config_sociedades.py`
2. Agregar un bloque a la lista `SOCIEDADES`
3. Guardar

Resultado automático:

- ☒ Descarga SENA la incluye
- ☒ Validador la reconoce
- ☒ Diálogos la muestran
- ☒ Detección automática funciona

Sin cambios en otros archivos - Sistema 100% escalable

Descarga Automática SENA

Flujo General de Descarga

1. Inicio desde Validador

```

Usuario → Click "Descargar Archivos SENA"
↓
Validador importa SOCIEDADES desde config_sociedades
↓
Detecta archivos existentes en Insumos/[Mes Año]/
↓
Pregunta individualmente por cada archivo existente
↓
Calcula archivos a procesar (nuevos + sobrescribir)
↓
Ejecuta: python descarga_sena.py --sobrescribir=Sociedad1 --
sobrescribir=Sociedad2

```

2. Ejecución en descarga_sena.py

```

# Parsea argumentos
ARCHIVOS_A_SOBRESCRIBIR = [] # Lista de sociedades a sobrescribir

# Itera SOCIEDADES
for sociedad in SOCIEDADES:
    nombre = sociedad["nombre"]

    # Verificación previa de archivo
    if archivo_existe(nombre):
        if nombre not in ARCHIVOS_A_SOBRESCRIBIR:
            return "OMITIDO" # Sin Login, sin descarga
        else:
            # Continuar con descarga (sobrescribir)

    # Descarga completa
    login(sociedad["usuario"], sociedad["contrasena"])
    descargar_reporte()
    renombrar_archivo(nombre)
    cerrar_sesion()

```

3. Carga Selectiva en Validador

```
descarga_sena.py termina
↓
Validador carga solo archivos_a_procesar:
- Archivos nuevos descargados
- Archivos sobrescritos
- NO carga archivos conservados
↓
Usuario continúa con validación
```

Gestión Inteligente de Archivos Existentes

Escenario de Ejemplo

Estado inicial:

- Bancolombia.xlsx → existe
- Fiduciaria.xlsx → existe
- Banca de Inversiones.xlsx → NO existe
- Valores.xlsx → NO existe
- Grupo CIBEST.xlsx → NO existe

Interacción con usuario:

```
¿Sobrescribir Bancolombia.xlsx? → Usuario: NO
¿Sobrescribir Fiduciaria.xlsx? → Usuario: SÍ
```

Resultado:

```
archivos_existentes = [Bancolombia, Fiduciaria]
archivos_a_sobrescribir = [Fiduciaria]
archivos_a_procesar = [Fiduciaria, Banca, Valores, CIBEST]
```

Descargas:

```
✓ Fiduciaria (sobrescrito)
✓ Banca de Inversiones
✓ Valores
✓ Grupo CIBEST
X Bancolombia (conservado - sin login)
```

Carga en validador:

```
✓ Fiduciaria.xlsx
✓ Banca de Inversiones.xlsx
✓ Valores.xlsx
✓ Grupo CIBEST.xlsx
X Bancolombia.xlsx (NO se carga - archivo antiguo conservado)
```

Optimización de Performance

Verificación previa (antes de iniciar navegador):

```

archivo_destino = Path(CARPETA_DESCARGAS) / f"{nombre_sociedad}.xlsx"
if archivo_destino.exists() and nombre_sociedad not in
ARCHIVOS_A_SOBREESCRIBIR:
    return "OMITIDO" # Retorno inmediato

# Si no retorna, continúa con navegador
driver = webdriver.Edge(...)

```

Ahorro de tiempo:

- Antes: ~10 segundos por archivo omitido (iniciar navegador + cerrar)
- Ahora: ~0.1 segundos (verificación de archivo)
- **Ejemplo:** 3 archivos omitidos = ahorro de ~30 segundos

Función: descargar_sociedad()

Ubicación: descarga_sena.py

Parámetros:

- nombre_sociedad: str
- usuario: str (NIT)
- contraseña: str

Retorna:

- WebDriver object: Si descarga exitosa
- "OMITIDO": Si archivo existe y no debe sobrescribir
- None: Si falla

Pasos detallados:

1. Verificación previa

```

if archivo_existe and not en_lista_sobrescribir:
    print(f"[INFO] Conservando archivo existente")
    return "OMITIDO"

```

2. Configuración de navegador

```

edge_options = Options()
edge_options.add_argument("--headless=new")           # Invisible
edge_options.add_argument("--disable-gpu")
edge_options.add_argument("--log-level=3")           # Mínimo Logging
edge_options.add_argument("--no-sandbox")
edge_options.add_argument("--disable-dev-shm-usage")

```

3. Login automático

- Navega a URL_SENA
- Localiza campos de usuario y contraseña
- Ingresa credenciales

- Click en botón login
- Verifica cambio de URL (hasta 3 reintentos)

4. Navegación al reporte

- Abre pestaña con URL de reportes
- Cambia a la pestaña
- Espera carga completa (document.readyState)
- Verifica presencia de botón "Consultar"

5. Descarga del archivo

- Click en botón "Consultar"
- Espera resultados
- Click en botón export Excel
- Espera descarga (detecta archivos .crdownload)
- Timeout configurable (TIMEOUT_DESCARGA)

6. Renombrado de archivo

```
def renombrar_archivo_sena(carpeta, nuevo_nombre):
    archivo_descargado = obtener_ultimo_archivo(carpeta)
    nuevo_path = carpeta / f"{nuevo_nombre}.xlsx"

    if nuevo_path.exists() and nuevo_nombre not in
ARCHIVOS_A_SOBREESCRIBIR:
        archivo_descargado.unlink() # Eliminar descargado
        return None # Conservar existente

    archivo_descargado.rename(nuevo_path)
```

7. Cierre de sesión

- Busca enlace de logout
- Click en cerrar sesión
- Cierra navegador

Reintentos Automáticos

Configuración: MAX_REINTENTOS_DESCARGA (desde config_sociedades.py)

Lógica:

```
for sociedad in SOCIEDADES:
    for intento in range(1, MAX_REINTENTOS_DESCARGA + 1):
        resultado = descargar_sociedad(...)

        if resultado == "OMITIDO":
            break # No reintentar archivos omitidos
        elif resultado: # Éxito
            break
        else: # Falló
            if intento < MAX_REINTENTOS_DESCARGA:
```

```
time.sleep(5)
continue # Reintentar
```

Supresión de Logs

Problema: Edge/Chromium genera warnings internos en consola

Solución:

```
# Argumentos del navegador
edge_options.add_argument("--log-level=3")
edge_options.add_argument("--disable-logging")

# Servicio del driver
service = Service(
    log_output=os.devnull,
    service_log_path=os.devnull
)
```

Resultado: Ejecución silenciosa sin mensajes de error internos

Dependencias y Conexiones

Bibliotecas Requeridas

validador_monetizacion.py:

```
import pandas as pd
from datetime import datetime
import os
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
import threading
import locale
from pathlib import Path
from config_sociedades import SOCIEDADES # Importación dinámica
```

descarga_sena.py:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.edge.service import Service
from selenium.webdriver.edge.options import Options
import time
import os
import sys
from pathlib import Path
from datetime import datetime
import locale
```



```
from config_sociudades import (
    SOCIEDADES,
    URL_SENA,
    MAX_REINTENTOS_DESCARGA,
    TIMEOUT_DESCARGA
)
```

Archivos de Entrada

1. Archivos SENA

Origen: Descargados automáticamente o cargados manualmente

Ubicación: Insumos/[Mes Año]/[Sociedad].xlsx

Campos Principales:

Campo	Descripción	Tipo	Uso
Documento	Identificación del aprendiz	Texto	Clave primaria
Nombre del aprendiz	Nombre completo	Texto	Identificación
Tipo aprendiz	Clasificación (Regular/BT)	Texto	Validación
Fecha inicio	Inicio de contrato	Fecha	Validación fechas
Fecha fin	Fin etapa lectiva	Fecha	Cálculo vigencia
Fecha fin productiva	Fin etapa productiva	Fecha	Priorizada
Estado del contrato	Estado actual	Choice	Validación estado
Sociedad	Código de sociedad	Texto	Agrupación

Detección automática de sociedad:

- Usa funciones de config_sociudades.py
- Busca palabras clave en nombre de archivo
- Asigna código interno correspondiente

2. Archivo SAP

Origen: Exportado desde sistema SAP

Campos Principales:

Campo	Descripción	Tipo	Uso
Número ID	Documento	Texto	Clave primaria
Nombre de pila	Primer nombre	Texto	Identificación
Apellido	Apellidos	Texto	Identificación

Campo	Descripción	Tipo	Uso
Desde	Fecha inicio	Fecha	Validación fechas
Fin contr.	Fecha fin	Fecha	Validación fechas
Status ocupación	Estado	Choice	Validación estado
Soc.	Código sociedad	Texto	Agrupación
Denominación de función	Cargo	Texto	Info adicional
Denominación de posiciones	Tipo aprendiz	Texto	Validación BT

3. Archivo RETIRO (Opcional)

Origen: Sistema de gestión de retiros

Campos Principales:

Campo	Descripción	Tipo	Uso
Número ID	Documento	Texto	Búsqueda
Desde	Fecha retiro	Fecha	Info retiro
Motivo de terminación del cont	Razón	Texto	Justificación
Relación laboral	Tipo	Texto	Filtro

4. Informe Anterior (Opcional)

Origen: Reporte generado en periodo anterior

Hoja: "Análisis"

Uso: Comparación y copia de observaciones

Clases Principales

Clase: ValidadorMonetizacion

Propósito: Lógica central de validación y procesamiento

Atributos Principales

```
self.archivos_sena_seleccionados = [] # Rutas archivos SENA
self.df_sena_consolidado = None # DataFrame consolidado
self.df_sap = None # DataFrame SAP
self.df_retiro = None # DataFrame RETIRO
self.df_informe_anterior = None # DataFrame periodo anterior
```

```
self.mes_reporte = None           # Mes del reporte (1-12)
self.anio_reporte = None         # Año del reporte
```

Métodos Clave

detectar_sociedad_archivo(nombre_archivo)

- Importa `obtener_sociedades_palabras_clave()` de config
- Busca palabras clave en nombre
- Retorna (codigo_interno, nombre_sociedad)

obtener_nombre_sociedad(codigo_sociedad)

- Importa `obtener_mapeo_codigos()` de config
- Traduce código a nombre descriptivo

descargar_archivos_sena()

- Funcionalidad central de descarga automática
- Importa SOCIEDADES de config_sociedades
- Detecta archivos existentes
- Pregunta individualmente por sobrescritura
- Calcula archivos a procesar
- Ejecuta `descarga_sena.py` con parámetros
- Carga selectivamente solo archivos procesados

Flujo interno de `descargar_archivos_sena()`:

```
# 1. Importar sociedades
from config_sociedades import SOCIEDADES
sociedades = SOCIEDADES

# 2. Detectar existentes
mes_anio = datetime.now().strftime("%B %Y").capitalize()
carpeta_destino = Path(__file__).parent / "Insumos" / mes_anio
archivos_existentes = [
    s["nombre"] for s in sociedades
    if (carpeta_destino / f"{s['nombre']}.xlsx").exists()
]

# 3. Preguntar individualmente
archivos_a_sobrescribir = []
for sociedad in archivos_existentes:
    respuesta = messagebox.askyesno(
        "Archivo existente",
        f"¿Sobrescribir {sociedad}.xlsx?"
    )
    if respuesta:
        archivos_a_sobrescribir.append(sociedad)
```

```

# 4. Calcular archivos a procesar
archivos_a_procesar = [
    s["nombre"] for s in sociedades
    if s["nombre"] not in archivos_existentes
    or s["nombre"] in archivos_a_sobrescribir
]

# 5. Ejecutar descarga
comando = ["python", "-u", "descarga_sena.py"]
for sociedad in archivos_a_sobrescribir:
    comando.append(f"--sobrescribir={sociedad}")

# 6. Carga selectiva
for nombre_sociedad in archivos_a_procesar:
    archivo_path = carpeta_destino / f"{nombre_sociedad}.xlsx"
    if archivo_path.exists():
        self.archivos_sena_seleccionados.append(str(archivo_path))

```

comparar_sap_vs_sena()

- Compara cada registro SAP contra SENA
- Detecta diferencias específicas
- Verifica estado en RETIRO
- Clasifica como justificada/no justificada

comparar_sena_vs_sap()

- Compara cada registro SENA contra SAP
- Identifica registros sin SAP
- Verifica estado en RETIRO

generar_hoja_analisis(df_sap_vs_sena, df_sena_vs_sap)

- Consolida diferencias de ambas comparaciones
- Identifica casos nuevos vs existentes
- Copia observaciones del informe anterior
- Genera resúmenes concisos de registros

generar_reporte_completo(ruta_salida)

- Crea archivo Excel con múltiples hojas
- Aplica formatos corporativos
- Ajusta anchos de columna
- Ver sección "Generación de Reportes"

Clase: InterfazValidador

Propósito: Interfaz gráfica de usuario

Atributos de Interfaz

```

self.validador = ValidadorMonetizacion()
self.ventana = tk.Tk()
self.archivos_sena_seleccionados = []
self.archivo_sap = None
self.archivo_retiro = None
self.archivo_informe_anterior = None

```

Secciones de la Interfaz

1. **Selector Mes/Año:** ComboBoxes para periodo del reporte
2. **Archivos SENA:**
 - Botón "Descargar Archivos SENA"
 - Botón "Agregar archivos manualmente"
 - ListBox con archivos cargados
 - Contador de archivos
3. **Archivo SAP:** Selector obligatorio
4. **Archivo RETIRO:** Selector opcional
5. **Informe Anterior:** Selector opcional (colapsable)
6. **Botones de Acción:** Generar/Limpiar
7. **Área de Mensajes:** Log con colores por tipo

Método: descargar_archivos_sena()

Funcionalidad de descarga automática

```

def descargar_archivos_sena(self):
    """Descarga automática con confirmación individual"""

    # Importar sociedades
    from config_sociedades import SOCIEDADES

    # Detectar existentes
    archivos_existentes = self._detectar_archivos_existentes(SOCIEDADES)

    # Pregunta inicial
    if not messagebox.askyesno("Confirmar", "¿Iniciar descarga?"):
        return

    # Preguntar por cada existente
    archivos_a_sobrescribir = []
    for sociedad in archivos_existentes:
        if messagebox.askyesno("Archivo existente",
                                f"¿Sobrescribir {sociedad}?"):
            archivos_a_sobrescribir.append(sociedad)

    # Calcular archivos a procesar
    archivos_a_procesar = self._calcular_archivos_a_procesar(
        SOCIEDADES,
        archivos_existentes,
        archivos_a_sobrescribir

```

```
)

# Ejecutar en hilo separado
threading.Thread(
    target=self._ejecutar_descarga,
    args=(archivos_a_sobrescribir, archivos_a_procesar),
    daemon=True
).start()
```

Estructura de Datos

DataFrames Principales

El sistema utiliza Pandas DataFrames para procesar y comparar información. Cada DataFrame tiene una estructura específica según su origen:

1. df_sena_consolidado

Propósito: Consolidación de todos los archivos SENA cargados

Estructura:

```
{
    'Documento': str,                # Número de identificación (clave
    primaria)
    'Nombre del aprendiz': str,      # Nombre completo del aprendiz
    'Tipo aprendiz': str,            # "Regular" o "BT" (Banca Tecnología)
    'Fecha inicio': datetime,        # Fecha de inicio del contrato
    'Fecha fin': datetime,           # Fecha fin etapa lectiva
    'Fecha fin productiva': datetime,# Fecha fin etapa productiva
    (priorizada)
    'Estado del contrato': str,      # "En formación", "Cancelado", etc.
    'Sociedad': str,                 # Código interno (1000, 3000, 4000,
    5000, 0100)
    'Archivo_Origen': str            # Nombre del archivo Excel original
}
```

Normalización aplicada:

- Conversión de fechas a formato datetime
- Eliminación de espacios en blanco
- Conversión de documento a string
- Asignación automática de código de sociedad según nombre de archivo

Uso: Base para comparación SENA vs SAP

2. df_sap

Propósito: Datos exportados del sistema SAP

Estructura:

```
{
  'Número ID': str,           # Documento (clave primaria)
  'Nombre de pila': str,      # Primer nombre
  'Apellido': str,            # Apellidos completos
  'Desde': datetime,          # Fecha inicio contrato
  'Fin contr.': datetime,      # Fecha fin contrato
  'Status ocupación': str,     # "Ocupado", "Activo", etc.
  'Soc.': str,                 # Código sociedad (1000, 3000,
etc.)
  'Denominación de función': str, # Cargo o función
  'Denominación de posiciones': str, # Tipo de posición
  'División': str,             # División organizacional
  'Descripción de división de personal': str # Descripción adicional
}
```

Normalización aplicada:

- Conversión de fechas a datetime
- Limpieza de espacios
- Conversión de documento a string
- Filtrado por periodo de vigencia

Uso: Base para comparación SAP vs SENA

3. df_retiro

Propósito: Registros de empleados retirados

Estructura:

```
{
  'Número ID': str,           # Documento
  'Desde': datetime,          # Fecha de retiro
  'Motivo de terminación del cont': str, # Razón del retiro
  'Relación laboral': str,     # Tipo de relación
  'Nombre de pila': str,       # Primer nombre
  'Apellido': str              # Apellidos
}
```

Filtrado aplicado:

- Solo registros con relación laboral "Practicante SENA"

Uso:

- Justificación de diferencias SAP-SENA
 - Identificación de retiros formales
-

4. df_informe_anterior

Propósito: Reporte del periodo anterior

Estructura (hoja "Análisis"):

```
{
  'Documento': str,          # Número de identificación
  'Nombre Completo': str,    # Nombre del aprendiz
  'Diferencia': str,         # Tipo de diferencia detectada
  'Observación': str        # Comentarios del analista
}
```

Uso:

- Copiar observaciones previas a casos recurrentes
 - Identificar casos nuevos vs existentes
 - Mantener historial de análisis
-

5. df_sap_vs_sena (resultado de comparación)

Propósito: Registros SAP que difieren o no existen en SENA

Estructura:

```
{
  # Datos SAP originales
  'Documento SAP': str,
  'Nombre SAP': str,
  'Sociedad SAP': str,
  'Fecha Inicio SAP': datetime,
  'Fecha Fin SAP': datetime,
  'Estado SAP': str,
  'Tipo Aprendiz SAP': str,

  # Datos SENA (si existen)
  'Nombre SENA': str,
  'Sociedad SENA': str,
  'Fecha Inicio SENA': datetime,
  'Fecha Fin SENA': datetime,
  'Estado SENA': str,
  'Tipo Aprendiz SENA': str,

  # Análisis
  'Diferencia Detectada': str,          # "Sociedad diferente", "Fecha diferente", etc.
  'En RETIRO': str,                    # "Sí" o "No"
  'Fecha RETIRO': datetime,
  'Motivo RETIRO': str,
  'Clasificación': str                 # "Justificada" o "No Justificada"
}
```

Tipos de diferencias detectadas:

- "No existe en SENA"

- "Sociedad diferente"
- "Fecha inicio diferente"
- "Fecha fin diferente"
- "Estado diferente"
- "Tipo aprendiz diferente"

6. df_sena_vs_sap (resultado de comparación)

Propósito: Registros SENA que no existen en SAP

Estructura:

```
{
  # Datos SENA completos
  'Documento': str,
  'Nombre del aprendiz': str,
  'Sociedad': str,
  'Fecha inicio': datetime,
  'Fecha fin': datetime,
  'Fecha fin productiva': datetime,
  'Estado del contrato': str,
  'Tipo aprendiz': str,
  'Archivo_Origen': str,

  # Verificación RETIRO
  'En RETIRO': str,          # "Sí" o "No"
  'Fecha RETIRO': datetime,
  'Motivo RETIRO': str
}
```

Uso: Identificar aprendices en SENA sin registro SAP

7. df_analisis (hoja principal del reporte)

Propósito: Vista consolidada para análisis manual

Estructura:

```
{
  'Documento': str,          # Número ID
  'Nombre Completo': str,    # Nombre del aprendiz
  'Diferencia': str,         # Tipo de diferencia
  'Registro SAP': str,       # Resumen conciso de datos SAP
  'Registro SENA': str,      # Resumen conciso de datos SENA
  'Caso Nuevo': str,        # "Sí" o "No"
  'Observación': str        # Campo editable para analista
}
```

Formato de resúmenes:

Registro SAP: "Bancolombia | 01/01/2025 - 31/12/2025 | En formación"

Registro SENA: "Fiduciaria | 01/02/2025 - 31/12/2025 | Activo"

Lógica de "Caso Nuevo":

- Compara documento con df_informe_anterior
- "Sí": Documento no existe en informe anterior
- "NO": Documento ya fue analizado previamente

Copia de observaciones:

- Si caso NO es nuevo, copia observación del informe anterior
 - Permite mantener continuidad en el análisis
-

Transformaciones de Datos

Normalización de fechas

```
def normalizar_fechas(df, columnas_fecha):  
    """Convierte columnas a datetime y maneja errores"""  
    for col in columnas_fecha:  
        if col in df.columns:  
            df[col] = pd.to_datetime(df[col], errors='coerce')  
    return df
```

Detección automática de sociedad

```
def detectar_sociedad_archivo(nombre_archivo):  
    """Detecta código de sociedad por nombre de archivo"""  
    from config_sociedades import obtener_sociedades_palabras_clave  
  
    sociedades = obtener_sociedades_palabras_clave()  
    nombre_lower = nombre_archivo.lower()  
  
    for palabra, (codigo, nombre) in sociedades.items():  
        if palabra in nombre_lower:  
            return codigo, nombre  
  
    return None, None
```

Filtrado por vigencia

```
def filtrar_por_vigencia(df, mes, anio):  
    """Filtra contratos vigentes en el periodo"""  
    primer_dia = datetime(anio, mes, 1)  
    ultimo_dia = datetime(anio, mes, calendar.monthrange(anio, mes)[1])  
  
    # Fecha inicio <= último día del mes  
    # Fecha fin >= primer día del mes
```

```
return df[
    (df['Fecha inicio'] <= ultimo_dia) &
    (df['Fecha fin'] >= primer_dia)
]
```

Funciones Core

Funciones de config_sociedades.py

obtener_mapeo_codigos()

```
def obtener_mapeo_codigos():
    """Retorna {codigo_interno: nombre_sociedad}"""
    return {s["codigo_interno"]: s["nombre"] for s in SOCIEDADES}
```

obtener_sociedades_palabras_clave()

```
def obtener_sociedades_palabras_clave():
    """Retorna {palabra_clave: (codigo, nombre)}"""
    resultado = {}
    for sociedad in SOCIEDADES:
        for palabra in sociedad["palabras_clave"]:
            resultado[palabra] = (
                sociedad["codigo_interno"],
                sociedad["nombre"]
            )
    return resultado
```

obtener_opciones_dialogo()

```
def obtener_opciones_dialogo():
    """Retorna [(nombre, codigo_interno)]"""
    return [(s["nombre"], s["codigo_interno"]) for s in SOCIEDADES]
```

set_periodo_reporte(mes, anio)

Propósito: Establece periodo para filtrar contratos vigentes

Lógica:

- Fecha inicio <= último día del mes
 - Fecha fin >= primer día del mes
-

Interfaz Gráfica

Paleta de Colores Corporativa

- Primario: #0033A0 (Azul Bancolombia)
- Secundario: #FDDA24 (Amarillo Bancolombia)

- Éxito: #28A745 (Verde)
- Error: #DC3545 (Rojo)
- Alerta: #FFA500 (Naranja)
- Info: #0078D4 (Azul información)

Botones en Sección SENA

Botón "Descargar Archivos SENA"

- Icono: 
- Color de fondo: #0078D4 (azul)
- Acción: Inicia proceso de descarga automática
- Tooltip: "Descarga automática desde portal SENA"

Botón "Agregar archivos manualmente"

- Icono: 
- Color de fondo: #6C757D (gris)
- Acción: Diálogo de selección de archivos
- Uso: Cuando ya tienes archivos descargados

Botón "Limpiar"

- Icono: 
 - Color de fondo: #DC3545 (rojo)
 - Acción: Elimina archivos de la lista
-

Flujo de Procesamiento

Flujo Completo con Descarga Automática

1. INICIO
 - Usuario abre validador
 - ↳ Interfaz se carga
 - ↳ Mes/Año actuales por defecto
2. DESCARGA AUTOMÁTICA
 - Usuario → Click "Descargar Archivos SENA"
 - ↳ Validador importa SOCIEDADES de config_sociedades.py
 - ↳ Detecta archivos existentes en Insumos/[Mes Año]/
 - └ Bancolombia.xlsx ✓ existe
 - └ Fiduciaria.xlsx ✓ existe
 - └ Otros X no existen
 - ↳ Pregunta individual por cada existente
 - └ "¿Sobrescribir Bancolombia?" → Usuario: NO

- └─ "¿Sobrescribir Fiduciaria?" → Usuario: SÍ
 - ↳ Calcula archivos a procesar
archivos_a_procesar = [Fiduciaria, Banca, Valores, CIBEST]
 - python descarga_sena.py --sobrescribir=Fiduciaria
 - ↳ descarga_sena.py procesa SOCIEDADES
Para cada sociedad:
 - └─ Bancolombia → existe, NO sobrescribir → OMITIDO (sin login)
 - └─ Fiduciaria → existe, SÍ sobrescribir → DESCARGA
 - └─ Banca → no existe → DESCARGA
 - └─ Valores → no existe → DESCARGA
 - └─ CIBEST → no existe → DESCARGA
 - ↳ Validador carga solo archivos_a_procesar
 - ✓ Fiduciaria.xlsx
 - ✓ Banca de Inversiones.xlsx
 - ✓ Valores.xlsx
 - ✓ Grupo CIBEST.xlsx
 - X Bancolombia.xlsx (NO - archivo antiguo conservado)
 - 3. CARGA MANUAL (Alternativa)
Usuario → Click "Agregar archivos manualmente"
 - ↳ Selecciona archivos ya descargados
 - ↳ Se agregan a la lista
 - 4. SELECCIÓN ARCHIVOS ADICIONALES
Usuario selecciona archivo SAP (obligatorio)
Usuario selecciona archivo RETIRO (opcional)
Usuario selecciona informe anterior (opcional)
 - 5. GENERACIÓN DE REPORTE
Usuario → Click "Generar Reporte"
 - ↳ Validación de campos obligatorios
 - ↳ Carga y normalización de datos
 - ↳ Comparaciones bidireccionales
 - ↳ Generación de hojas Excel
 - ↳ Aplicación de formatos
 - ↳ Guardado de archivo
 - 6. FINALIZACIÓN
Reporte generado exitosamente
 - ↳ Mensaje de confirmación
 - ↳ Archivos listos para análisis
-

Generación de Reportes

Estructura del Excel Generado

Hoja 1: SAP vs SENA

- Registros SAP que difieren o no existen en SENA
- Columnas: 18 campos incluyendo clasificación y diferencias
- Formato: Encabezado azul, fondo amarillo en tabla

Hoja 2: SENA vs SAP

- Registros SENA que no existen en SAP
- Columnas: 13 campos con info RETIRO
- Formato: Similar a SAP vs SENA

Hoja 3: Análisis

- Vista resumida para análisis manual
- Columnas optimizadas:
 - Documento
 - Nombre Completo
 - Diferencia
 - Registro SAP (resumen conciso)
 - Registro SENA (resumen conciso)
 - Caso Nuevo (SÍ/NO)
 - Observación (editable)

Hoja 4: Resumen

- Métricas ejecutivas:
 - Total registros SAP/SENA
 - Total diferencias
 - Registros en RETIRO
 - Fecha de validación

Hoja 5: Validaciones Adicionales

- Duplicados detectados
- Fechas inválidas
- Incoherencias con RETIRO

Hoja 6: SENA Consolidado

- Todos los registros SENA consolidados
- Para referencia y análisis

Hoja 7: SAP Original

- Datos SAP sin modificar

- Para referencia
-

Mantenimiento y Extensibilidad

Agregar Sociedad

Paso único: Editar `config_sociudades.py`

```
SOCIEDADES = [  
    # ... sociedades existentes  
    {  
        "nombre": "Nueva Filial",  
        "usuario": "999888777",  
        "contrasena": "ClaveNueva2026",  
        "codigo_interno": "7000",  
        "palabras_clave": ["nueva", "filial", "nf"]  
    }  
]
```

Automáticamente sincroniza:

- ☒ Descarga SENA (6 sociedades)
- ☒ Diálogos de selección (6 botones)
- ☒ Detección automática (palabras: nueva, filial, nf)
- ☒ Validación de códigos (7000 → "Nueva Filial")
- ☒ Pregunta de sobrescritura

Sin cambios en otros archivos

Modificar Credenciales

```
# En config_sociudades.py  
{  
    "nombre": "Bancolombia",  
    "usuario": "890903938",  
    "contrasena": "NuevaContrasena2026", # ← Solo cambia aquí  
    ...  
}
```

Ajustar Timeouts

```
# En config_sociudades.py  
MAX_REINTENTOS_DESCARGA = 5      # Cambiar de 3 a 5  
TIMEOUT_DESCARGA = 120          # Cambiar de 60 a 120 segundos
```

Agregar Palabras Clave

```
# Si archivos no se detectan automáticamente  
{
```

```
    "nombre": "Bancolombia",  
    "palabras_clave": ["bancolombia", "banco", "col", "bco"] # Más  
variantes  
}
```

Troubleshooting

Problemas Comunes

Error: "No se encontraron archivos para cargar"

- Causa: Todos conservados, ninguno descargado
- Solución: Normal - archivos antiguos mantenidos

Error: Selenium no encuentra Edge

- Causa: Edge no instalado o ruta incorrecta
- Solución: Instalar Edge o verificar PATH

Error: "charmap codec can't decode"

- Causa: Problemas encoding Windows
- Solución: Ya corregido con UTF-8 + errors='replace'

Error: Portal SENA no responde

- Causa: Portal caído o mantenimiento
- Solución: Reintentar más tarde

Debugging

Mensajes de debug en validador:

```
[DEBUG] Archivos existentes: Bancolombia, Fiduciaria  
[DEBUG] Buscando en: c:\...\Insumos\Enero 2026
```

Logs en descarga_sena.py:

```
[INFO] Conservando archivo existente para Bancolombia  
[ADVERTENCIA] Sobrescribiendo archivo de Fiduciaria
```

Requisitos del Sistema

Software

- Python 3.13+
- Microsoft Edge (para descarga automática)
- Windows 10/11

Bibliotecas Python

```
pip install pandas xlswriter selenium
```
